Embeddings

Ciencia de Datos - FIUBA

Lic. Mateo Suster

Sobre el Problema

- El lenguaje natural es un sistema complejo utilizado para expresar significados. En este sistema, las palabras son la unidad básica del significado.
- Por su parte, sabemos que los modelos de ciencia de datos no trabajan procesando el texto de manera automática, sino que realizan sus optimizaciones a partir de valores numéricos.
- Ahora bien, el problema reside en cómo asignarle un valor numérico a las palabras

Embeddings de palabras

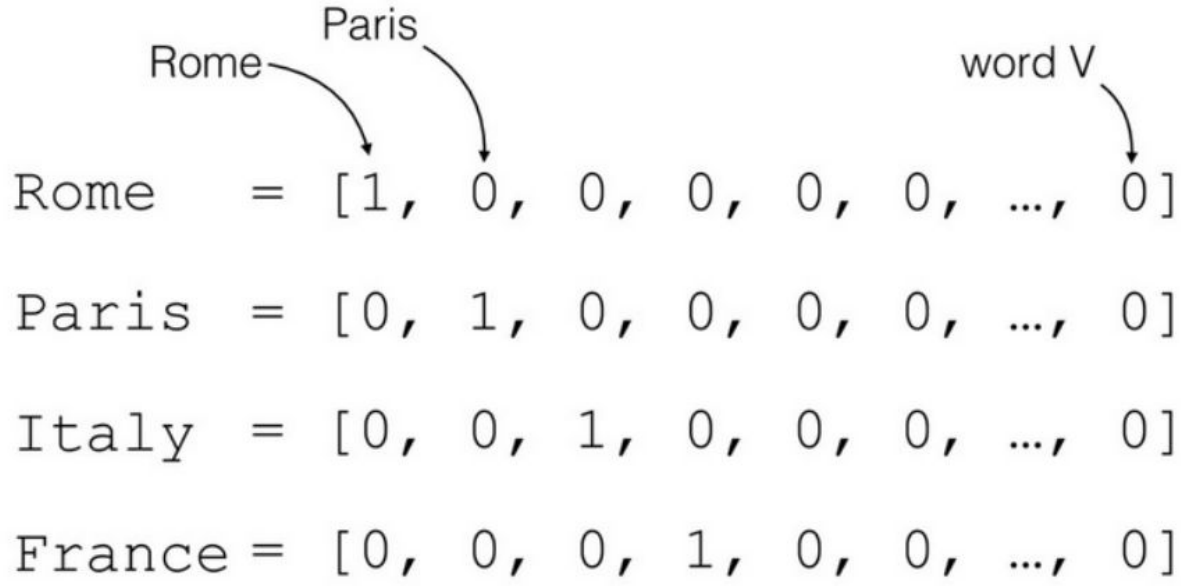
- Como su nombre lo indica, los **vectores de palabras** son vectores que se utilizan para representar palabras y también se pueden considerar como vectores de características o representaciones de palabras.
- La técnica de mapear palabras a vectores reales se llama **Embedding de palabras**. En los últimos años, los embedding de palabras se han convertido gradualmente en el conocimiento básico del procesamiento del lenguaje natural.

Algunas preguntas importantes

- ¿Cómo asignarle un valor numérico a las palabras?
- ¿Hay maneras más inteligentes que otras de codificar las palabras?
- ¿Hay forma de capturar el significado de las palabras dentro de los números que las identifican?

One-Hot encoding

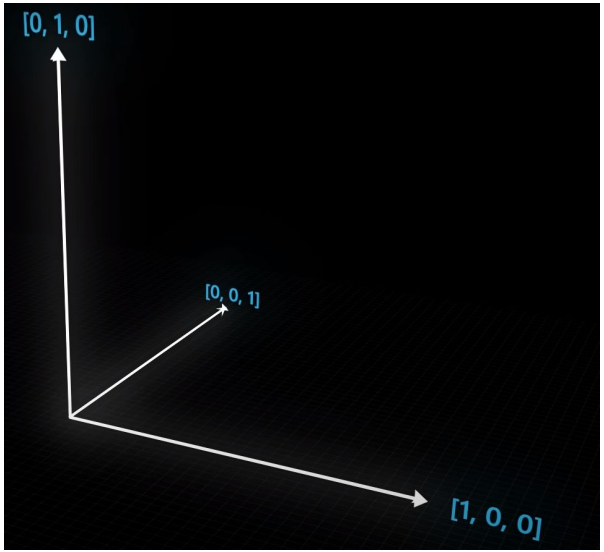
- Suponga que el número de palabras diferentes en el diccionario (su tamaño) es *N*, y cada palabra corresponde a un número entero diferente (índice) de *0* a *N-1*.
- Para obtener la representación vectorial one-hot para cualquier palabra con índice *i*, creamos un vector de longitud *N* con todos *0* y establecemos el elemento en la posición *i* a *1*.
- De esta manera, cada palabra se representa como un vector de longitud *N*, y puede ser utilizado directamente por redes neuronales.



One-Hot encoding

One-Hot encoding:

- [0] *1ª palabra*
- [0] *2ª palabra*
- [0] *3ª palabra*
- [1] ***4ª palabra***
- [0] *5ª palabra*
- [0] *6ª palabra*
- [0] *7ª palabra*
- [0] *8ª palabra*
- [0] *9ª palabra*
- [0] ...



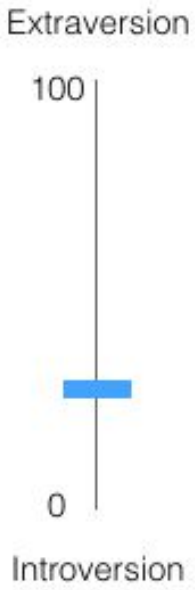
Todas las palabras son ortogonales, es decir están a la misma distancia o, lo que es lo mismo, consideradas como iguales

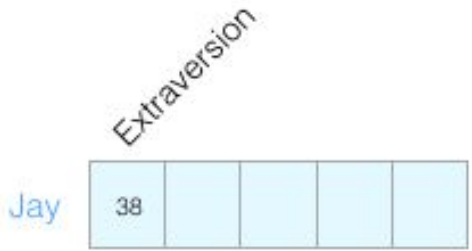
Los vectores One-Hot son una mala elección

- Aunque los vectores one-hot son fáciles de construir, por lo general no son una buena opción.
- Las principales razones son:
  - **Ineficiencia**: El tamaño de cada vector depende del tamaño del vocabulario. Dado que los vocabularios contienen varios cientos de miles de palabras, estos vectores ocupan mucho espacio y están mayormente compuestos de ceros.
  - **Ortogonalidad**: Cualquier par de vectores one-hot es ortogonal, por lo tanto no hay una noción natural de similaridad entre ellos. Por lo tanto, cada vector tiene información en dimensiones distintas y no se puede codificar la similaridad semántica entre palabras.

Ejemplo inspirador: Embedding de personalidad

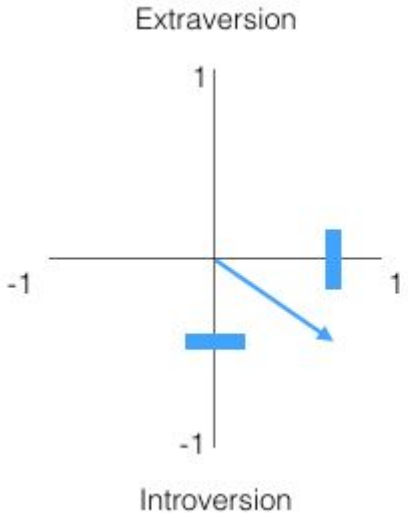
- En una escala de 0 a 100, ¿qué tan introvertido/extrovertido eres (donde 0 es el más introvertido y 100 es el más extrovertido)?
- ¿Alguna vez has realizado una prueba de personalidad como la prueba de los Cinco Grandes Rasgos de la Personalidad? Si no lo has hecho, estas son pruebas que te hacen una lista de preguntas, y luego te califican en una serie de ejes, siendo la introversión/extraversión uno de ellos.
- Imaginemos que Jay obtuvo una puntuación de 38/100 en el rasgo introversión/extraversión. Podemos graficar eso de la siguiente manera.

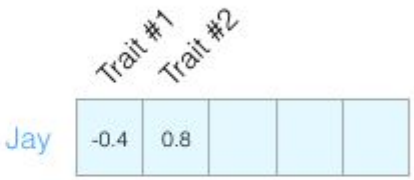




Ejemplo inspirador: Embedding de personalidad

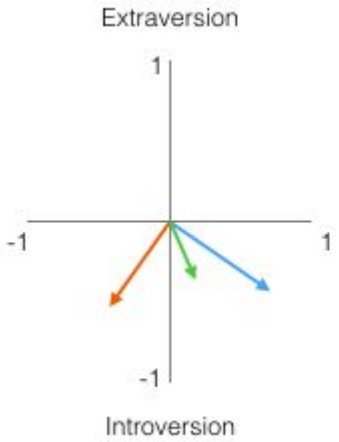
- Supongamos que, además de ese, también contemplamos otro de los 5 rasgos y que las escalas de 0 a 100 son reemplazadas por una escala de -1 a 1.
- Así obtendríamos un vector que represente a Jay de la siguiente manera:





Ejemplo inspirador: Embedding de personalidad

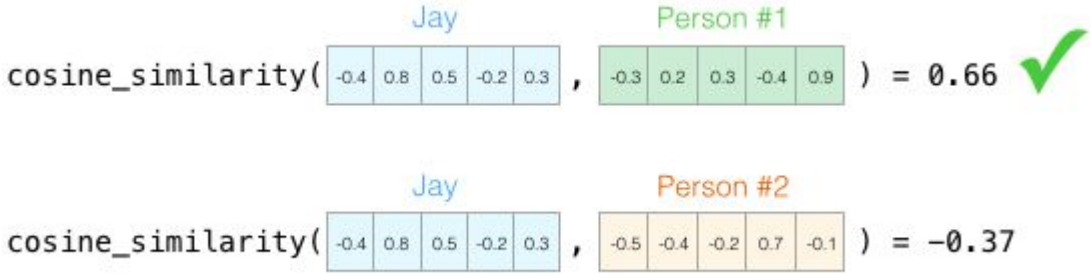
- Ahora podemos decir que este vector representa parcialmente la personalidad de Jay.
- La utilidad de tal representación viene cuando compararlo con otras dos personas.
- Digamos que lo atropella un autobús y necesita ser reemplazado por alguien con una personalidad similar. En la siguiente figura, ¿cuál de las dos personas se parece más a Jay?





Ejemplo inspirador: Embedding de personalidad

- Podemos ver que la persona n° 1 es más parecida a Jay porque gráficamente su vector está más "cerca" al de Jay. Ahora supongamos que tenemos un vector con los 5 rasgos... el análisis gráfico no es una opción.
- Sin embargo, podemos calcular la similaridad a través del ángulo entre los vectores (que se calcula utilizando el coseno como ya vimos) para obtener una métrica que facilite la comparación.



Seguimos viendo que la persona más parecida es la n°1, pero ahora tenemos una métrica para saber cuán mejor reemplazo es con respecto a la otra persona.

¿Cómo codificamos el significado?

- Para que las palabras sean procesadas por modelos de aprendizaje automático, necesitan alguna forma de representación numérica que los modelos puedan usar en sus cálculos.
- Podemos **usar un vector para representar correctamente las palabras** de una manera que **capture las relaciones semánticas** o de significado.
- Por ejemplo, al analizar estos vectores podemos saber si las palabras son **similares** u **opuestas** (analizando la dirección y el sentido del vector), o que un par de palabras como “Estocolmo” y “Suecia” tienen entre ellas la misma relación que tienen “El Cairo” y “Egipto” (analizando las distancias entre los vectores), o también **relaciones sintácticas** o basadas en la **gramática** (por ejemplo, la relación entre “tenía” y “tengo” es la misma que entre “era” y “es”).

¿Cómo codificamos el significado?

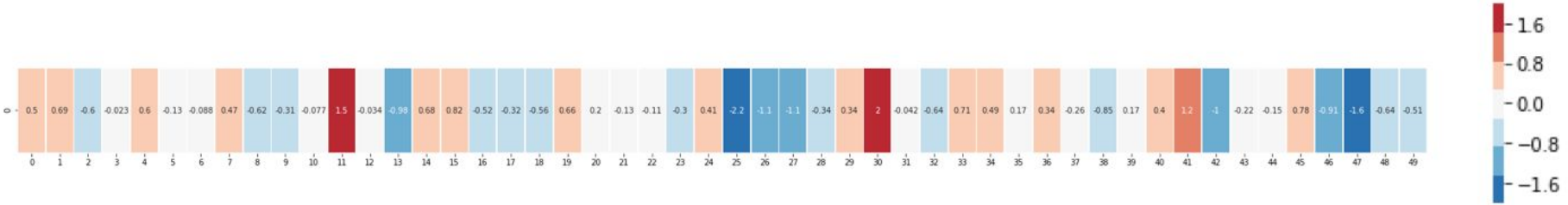
- Este es un embedding para la palabra "rey" (vector GloVe entrenado en Wikipedia)

$$[0.50451, 0.68607, -0.59517, -0.022801, 0.60046, -0.13498, -0.08813, 0.47377, -0.61798, -0.31012, -0.076666, 1.493, -0.034189, -0.98173, 0.68229, 0.81722, -0.51874, -0.31503, -0.55809, 0.66421, 0.1961, -0.13495, -0.11476, -0.30344, 0.41177, -2.223, -1.0756, -1.0783, -0.34354, 0.33505, 1.9927, -0.04234, -0.64319, 0.71125, 0.49159, 0.16754, 0.34344, -0.25663, -0.8523, 0.1661, 0.40102, 1.1685, -1.0137, -0.21585, -0.15155, 0.78321, -0.91241, -1.6106, -0.64426, -0.51042]$$

- Es una lista de 50 números. No podemos decir mucho mirando los valores.
- Pero visualicemos al mismo para poder compararlo con otros vectores de palabras.

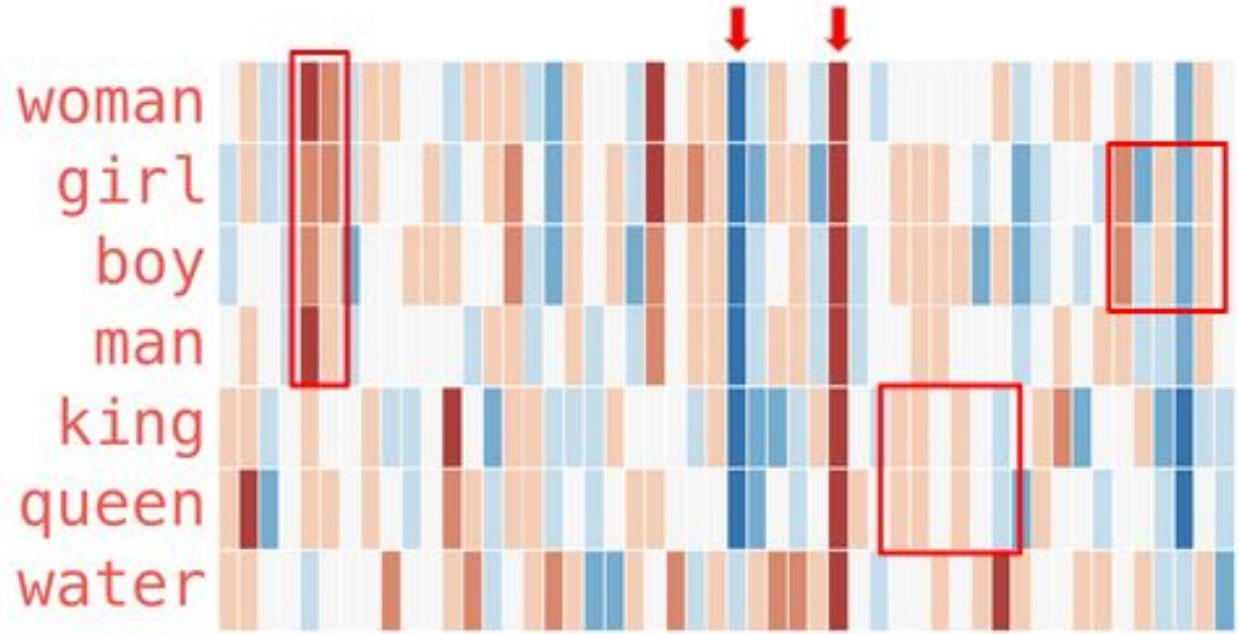
¿Cómo codificamos el significado?

- Pongamos todos estos números en una fila y codifiquemos por colores las celdas según sus valores (rojo si están cerca de 2, blanco si están cerca de 0, azul si están cerca de -2)
- Procederemos ignorando los números y solo observando los colores para indicar los valores de las celdas:



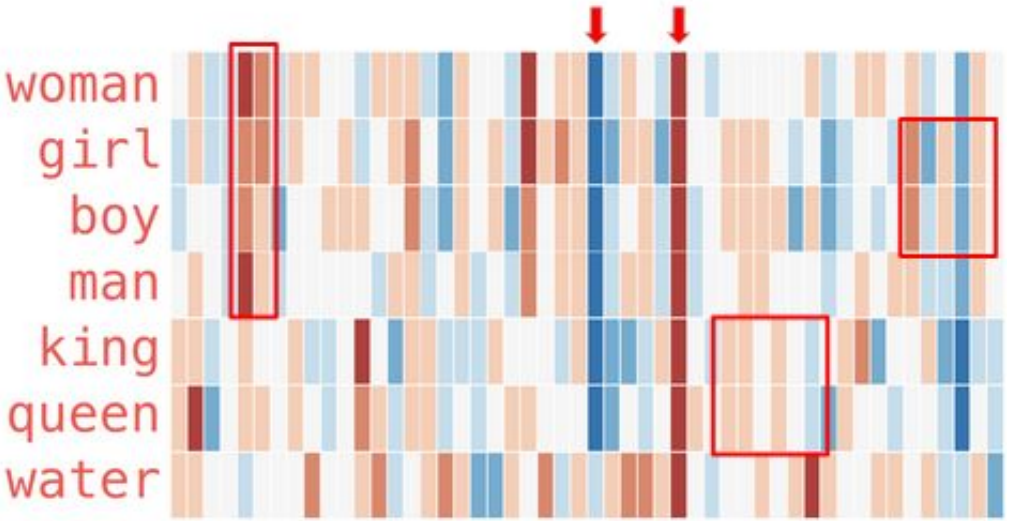
Visualización de Embeddings

- Ahora comparemos "Rey" con otras palabras:



Visualización de Embeddings

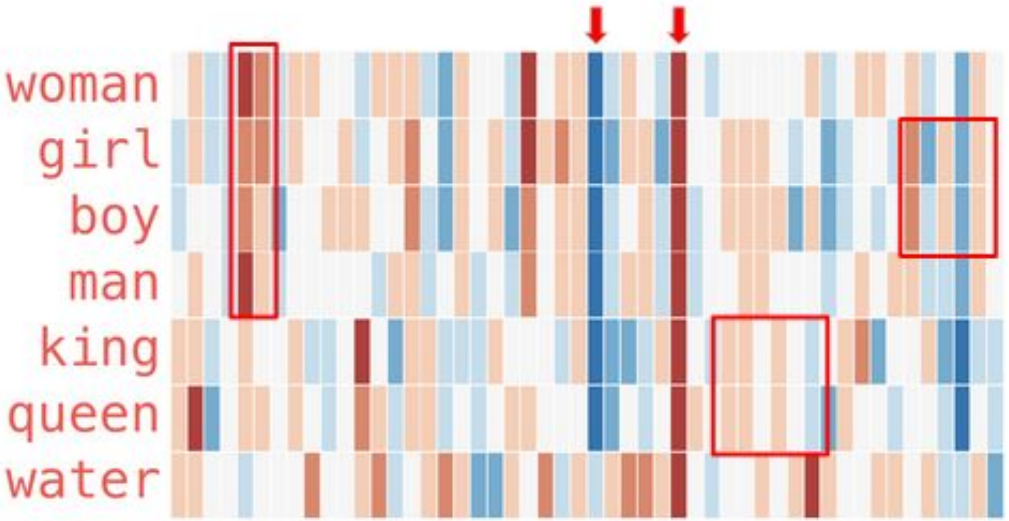
- Algunas cosas para señalar:



- Hay una columna roja recta a través de todas estas palabras diferentes. Son similares a lo largo de esa dimensión (y no sabemos qué codifica cada dimensión)
- Todas menos la última palabra son palabras que representan personas. Podemos, por ejemplo, ver que la columna azul va hacia abajo y se detiene antes del embedding de "agua".

Visualización de Embeddings

- Algunas cosas para señalar:



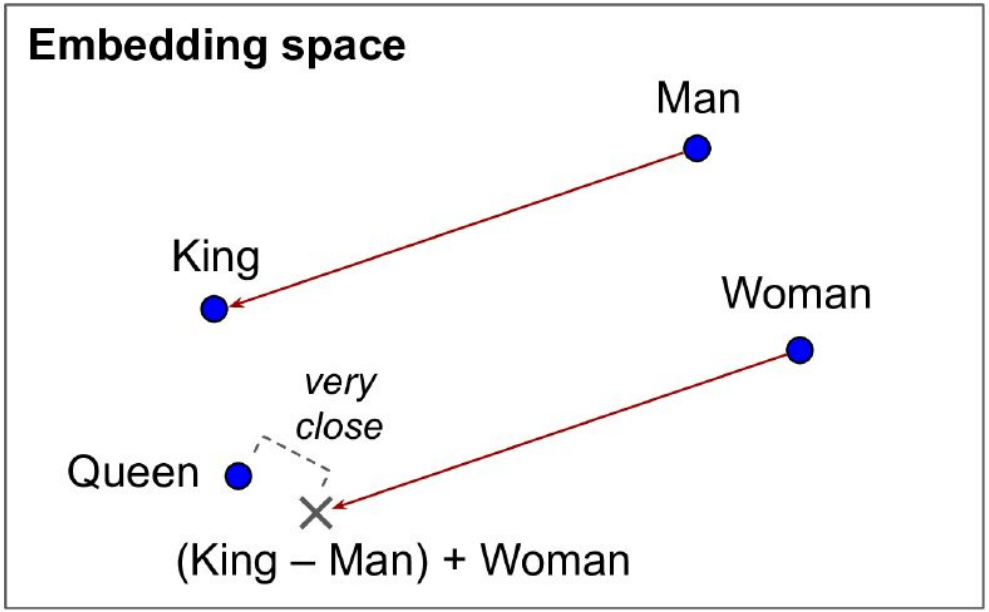
- Puedes ver cómo "mujer" y "niña" son similares entre sí en muchos lugares. Lo mismo con "hombre" y "niño"
- “niño” y “niña” también tienen lugares donde son similares entre sí, pero diferentes de “mujer” u “hombre”. ¿Podrían estar codificando una vaga concepción de la juventud?
- Hay lugares claros donde "rey" y "reina" son similares entre sí y distintos de todos los demás. ¿Podrían estar codificando un concepto vago de realeza?

Analogías de embeddings

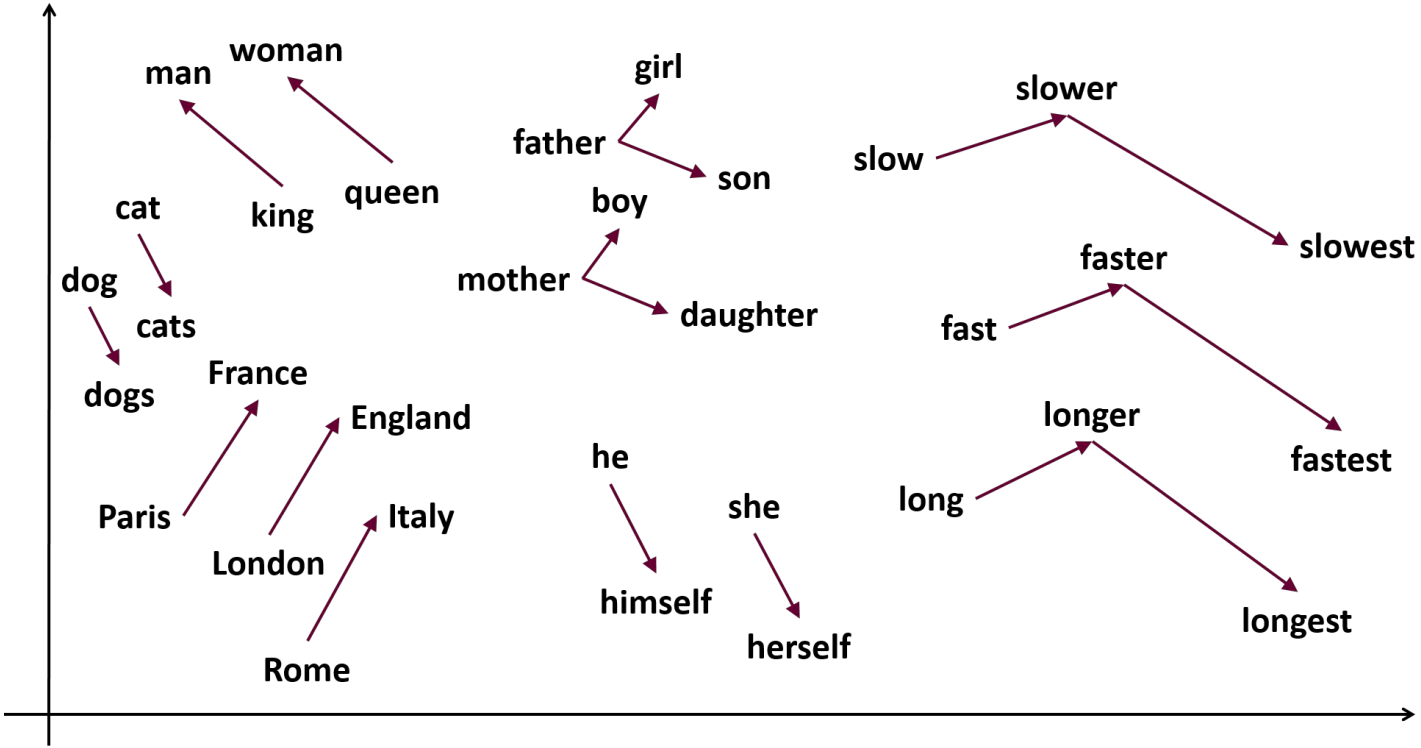
- Podemos sumar y restar embeddings de palabras y llegar a resultados interesantes. El ejemplo más famoso es la fórmula: "rey" - "hombre" + "mujer":
- El vector resultante de "rey-hombre+mujer" no es exactamente igual a "reina", pero "reina" es la palabra más cercana de los 400 000 embeddings de palabras que tenemos en esta colección.



Analogías de embeddings



Analogías de embeddings



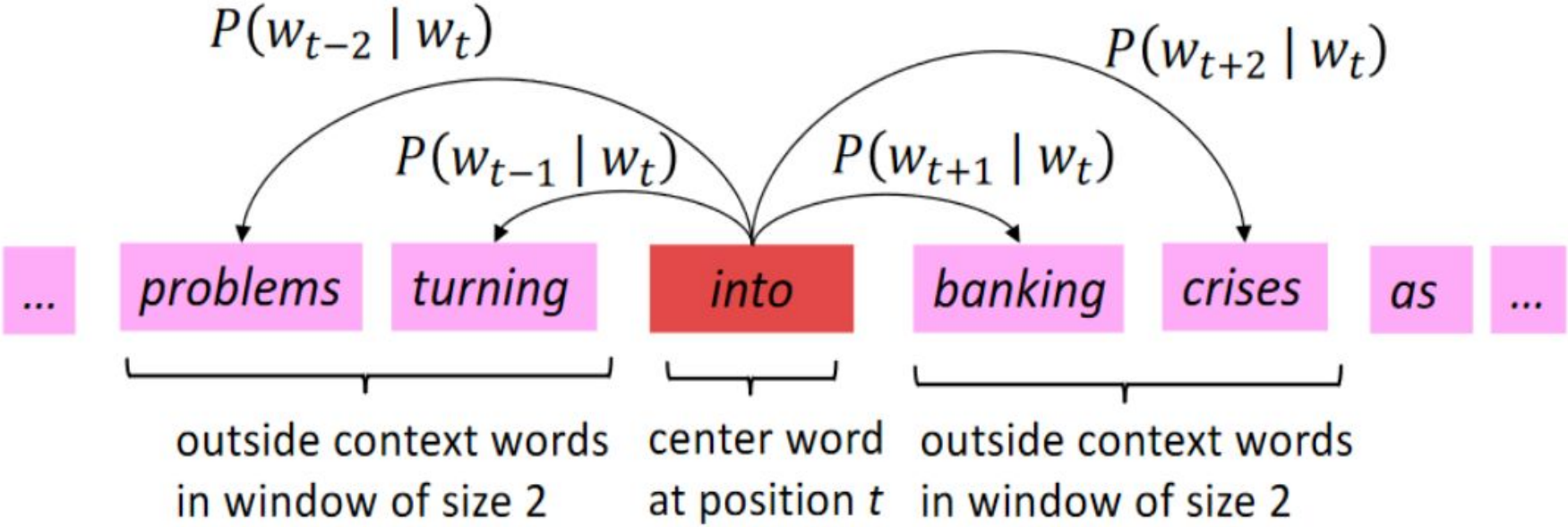
Sobre lo que representa cada dimensión

- Algo importante que debe quedar claro es que, en contraste con los embeddings de personalidad que analizamos anteriormente, **no sabemos que codifica cada dimensión del vector**.
- Podemos hacer conjeturas y experimentos que nos permitan validar que estos vectores están modelando de alguna u otra forma la semántica, pero esta codificación de los datos es la que a la red le pareció la más eficiente durante el entrenamiento.

Semántica distribucional

- La semántica distribucional es un área de investigación que desarrolla y estudia las teorías y métodos para **cuantificar y categorizar las similitudes semánticas entre elementos lingüísticos**, según sus *propiedades distribucionales* en grandes muestras de datos lingüísticos.
- La idea básica de la semántica distribucional se puede resumir en la llamada hipótesis distribucional: «**elementos lingüísticos con distribuciones similares tienen significados similares**».
- La hipótesis distribucional se origina de la teoría semántica del uso lingüístico, es decir, palabras que se usan y aparecen en los mismos contextos tienden a transmitir significados parecidos. La idea subyacente es "*dime con quién andas y te diré qué significas*".

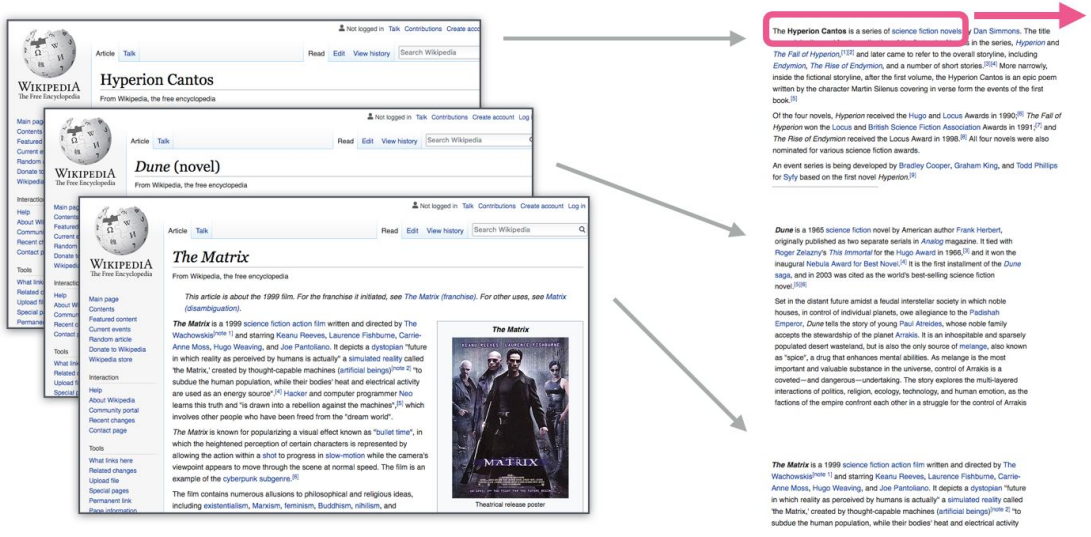
Semántica distribucional



Semántica distribucional

El uso de la semántica distribucional para entrenar modelos de lenguaje tiene como ventaja, frente a la mayoría de los otros modelos de aprendizaje automático, que **podemos entrenarlos con texto corriente**, el cual tenemos en abundancia, ya que la información que **necesitamos es la frecuencia con las que las palabras suelen aparecer juntas**.

Piensen en todos los libros, artículos, contenido de Wikipedia y otras formas de datos de texto que tenemos por ahí. Comparen esto con muchos otros modelos de aprendizaje automático que necesitan funciones hechas a mano y datos recopilados especialmente.



word2vec

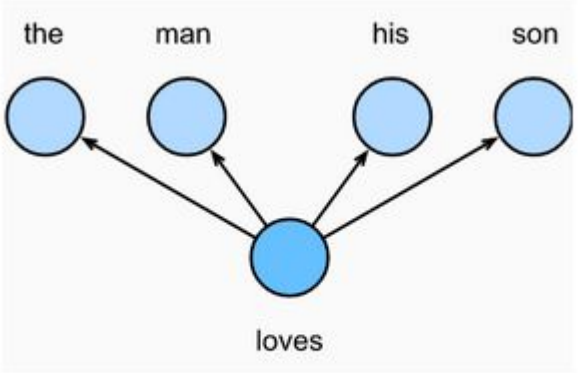
- La herramienta word2vec contiene dos modelos, a saber, **skip-gram** y **continuous bag of words (CBOW).**
- Para las representaciones semánticamente significativas, su entrenamiento se basa en **probabilidades condicionales** que pueden verse como la **predicción de las palabras usando algunas de las palabras que las rodean** en los corpus.
- Dado que la supervisión proviene de los datos sin etiquetas, tanto skip-gram como (CBOW) son **modelos autosupervisados.**

A continuación, presentaremos estos dos modelos y sus métodos de entrenamiento.

Skip-gram

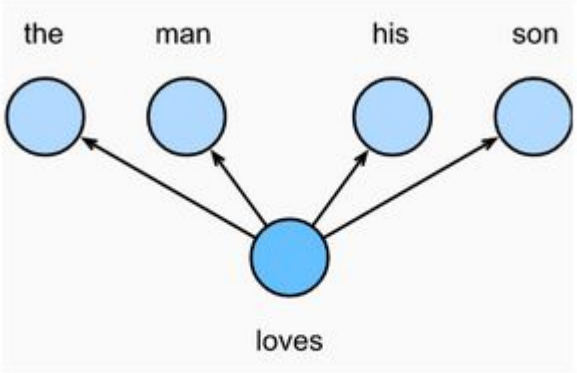
El modelo skip-gram asume que **una palabra** puede usarse para **generar las palabras que la rodean** en una secuencia de texto.

Considera la **probabilidad condicional** de generar las **palabras de contexto** que son aquellas que no están a más de 2 palabras de distancia de la **palabra central**.



$$P(\text{"the"}, \text{"man"}, \text{"his"}, \text{"son"} \mid \text{"loves"})$$

Skip-gram



$$P(\text{"the"}, \text{"man"}, \text{"his"}, \text{"son"} \mid \text{"loves"})$$

Suponga que las palabras de contexto se generan de forma independiente dada la palabra central (es decir, independencia condicional). En este caso, la probabilidad condicional anterior se puede reescribir como

$$P(\text{"the"} \mid \text{"loves"}) \cdot P(\text{"man"} \mid \text{"loves"}) \cdot P(\text{"his"} \mid \text{"loves"}) \cdot P(\text{"son"} \mid \text{"loves"}).$$

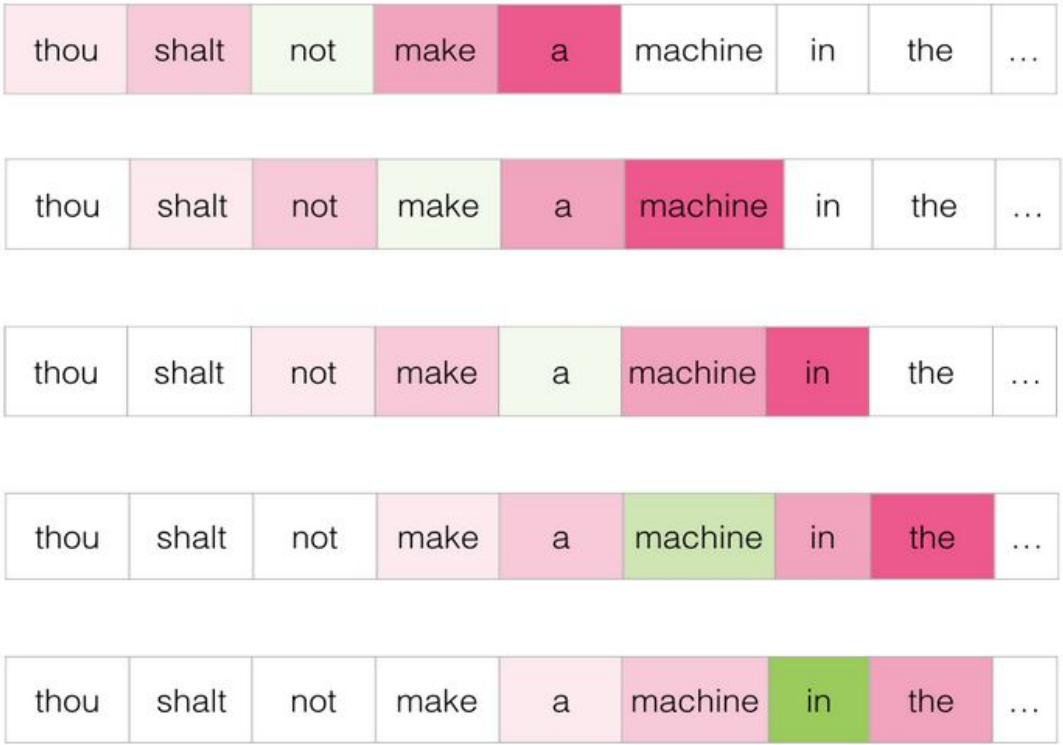
Skip-gram

- En el modelo skip-gram, cada palabra tiene dos vectores *d*-dimensionales asociados para calcular probabilidades condicionales. Más concretamente, para cualquier palabra con índice *i* en el diccionario, se denota por $\mathbf{v}_i \in \mathbb{R}^d$ y $\mathbf{u}_i \in \mathbb{R}^d$ sus dos vectores cuando se usan como una **palabra central** y una **palabra contexto**, respectivamente.
- La probabilidad condicional de generar cualquier palabra de contexto w_o (con índice *o* en el diccionario) dada la palabra central w_c (con índice *c* en el diccionario) se puede modelar mediante una operación softmax sobre el producto punto de los vectores:

$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}$$

donde se define el conjunto de índices de vocabulario $V=\{0,1,\ldots,|V|-1\}$

Skip-gram



input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Skip-gram

Dada una **secuencia de texto** de longitud *T*, donde la palabra en el paso de tiempo *t* se denota como $w^{(t)}$, suponga que las palabras de contexto se generan de forma independiente dada cualquier palabra central.

Para el **tamaño de ventana de contexto** *m*, la función de probabilidad del modelo skip-gram es **la probabilidad de generar todas las palabras de contexto dada cualquier palabra central**:

$$\prod_{t=1}^{T} \prod_{-m \leq j \leq m,\, j \neq 0} P(w^{(t+j)} \mid w^{(t)})$$

donde se puede omitir cualquier paso de tiempo que sea inferior a *1* ó superior a *T*

Es decir, tenemos dos productorias: del tamaño de la ventana de contexto *m* y del tamaño de la oración *T*

Skip-gram - optimización

- Los **parámetros del modelo en skip-gram son los vectores** que representan tanto a las palabras centrales como a las de contexto.
- Nuestro objetivo es encontrar estas representaciones vectoriales tratando de **maximizar la función de verosimilitud** (en inglés likelihood). En otras palabras, esta función se comporta como una función de pérdida, la cual va a tener un valor distinto según los θ elegidos para codificar nuestros embeddings.

$$L(\theta) = \prod_{t=1}^{T} \prod_{-m \leq j \leq m,\ j \neq 0} P(w^{(t+j)} \mid w^{(t)}; \theta)$$

Skip-gram - optimización

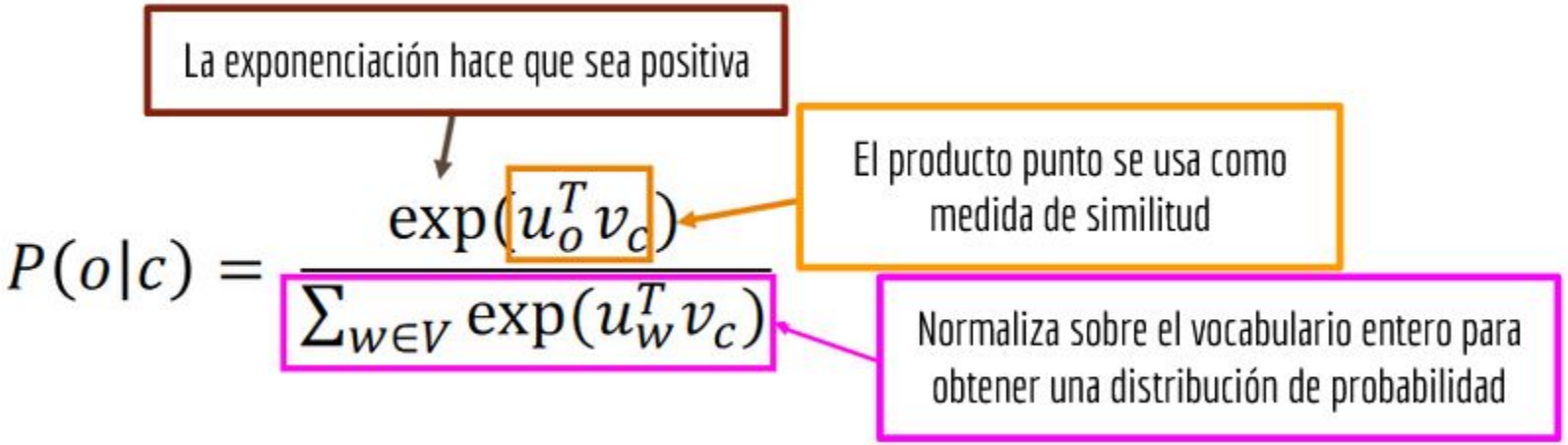
- Los propios embeddings (nuestros parámetros) se irán mejorando a medida que comparemos las salidas contra las etiquetas.
- Para transformar la función anterior en una **función de pérdida** que se debe minimizar, vamos a aplicarle el logaritmo, cambiarle el signo y calcular su valor promedio transformándola en la **verosimilitud logarítmica negativa media**

$$J(\theta) = -\frac{1}{T}\log L(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m\leq j\leq m,\, j\neq 0} \log P(w^{(t+j)} \mid w^{(t)};\theta)$$

- Para poder calcular esta función objetivo necesitamos calcular $P(w^{(t+j)} \mid w^{(t)};\theta)$ para lo cual vamos a usar dos vectores por cada palabra w:
  - v_w cuando w es la palabra central
  - u_w cuando w es una palabra de contexto

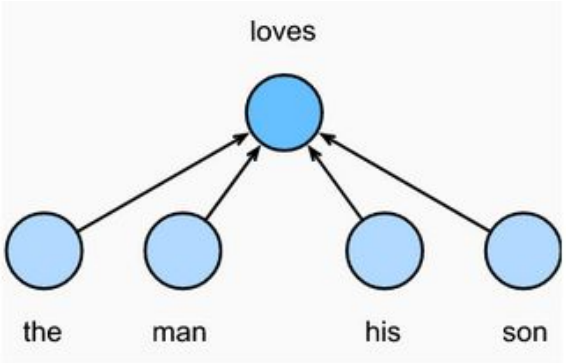
Skip-gram

De esta manera la probabilidad de que aparezca una palabra de contexto (*o*) dada una palabra central (*c*) es:



Continuous Bag of Words (CBOW)

El modelo *continuous bag of words* (CBOW) es similar al modelo skip-gram. La principal diferencia es que CBOW supone que **se genera una palabra central en función de las palabras de contexto** que la rodean en la secuencia de texto.



$$P(\text{"loves"} \mid \text{"the"}, \text{"man"}, \text{"his"}, \text{"son"})$$

Continuous Bag of Words (CBOW)

- Dado que hay múltiples palabras de contexto CBOW, estos vectores de palabras de contexto **se promedian** en el cálculo de la probabilidad condicional.
- Específicamente, para cualquier palabra con el índice i en el diccionario, se denota por $\mathbf{v}_i \in \mathbb{R}^d$ y $\mathbf{u}_i \in \mathbb{R}^d$ sus dos vectores cuando se usan como una **palabra de contexto** y una **palabra central** respectivamente (los significados se intercambian en el modelo skip-gram).
- La probabilidad condicional de generar cualquier palabra central w_c (con índice c en el diccionario) dadas las palabras de contexto que la rodean $w_{o_1},\ldots,w_{o_{2m}}$ (con índice $o_1,\ldots,o_{2m}$ en el diccionario) puede ser modelado por

$$P(w_c \mid w_{o_1}, \ldots, w_{o_{2m}}) = \frac{\exp\left(\frac{1}{2m}\mathbf{u}_c^\top (\mathbf{v}_{o_1} + \ldots + \mathbf{v}_{o_{2m}})\right)}{\sum_{i \in \mathcal{V}} \exp\left(\frac{1}{2m}\mathbf{u}_i^\top (\mathbf{v}_{o_1} + \ldots + \mathbf{v}_{o_{2m}})\right)}$$

Continuous Bag of Words (CBOW)

- Para abreviar, sea $W_o=\{w_{o1},\ldots,w_{o2m}\}$ el conjunto de las palabras de contexto en la ventana actual y $\bar{v}_o=(v_{o1}+\ldots+v_{o2m})/(2m)$ el vector promedio obtenido a partir de esas palabras. Entonces la ecuación anterior se puede simplificar como

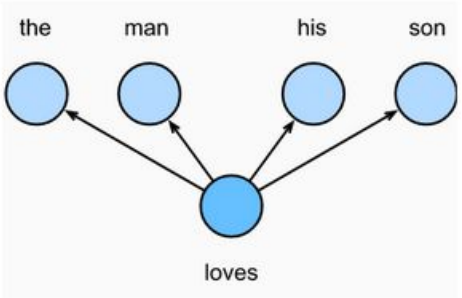
$$P(w_c \mid \mathcal{W}_o) = \frac{\exp\left(\mathbf{u}_c^\top \bar{\mathbf{v}}_o\right)}{\sum_{i\in\mathcal{V}} \exp\left(\mathbf{u}_i^\top \bar{\mathbf{v}}_o\right)}$$

- Dada una secuencia de texto de longitud T, donde la palabra en el paso de tiempo t se denota como $w^{(t)}$. Para el tamaño de la ventana de contexto m, la función de verosimilitud de CBOW es la probabilidad de generar todas las palabras centrales dadas sus palabras de contexto:

$$\prod_{t=1}^{T} P(w^{(t)} \mid w^{(t-m)},\ldots,w^{(t-1)},w^{(t+1)},\ldots,w^{(t+m)})$$

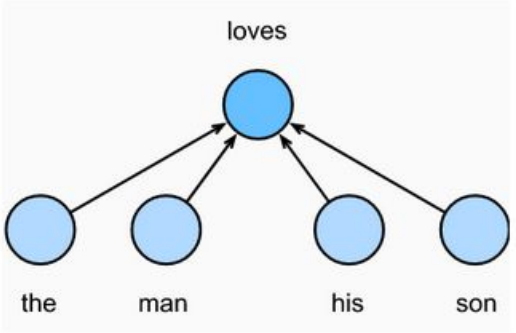
Comparación entre métodos

Skipgram



Skipgram necesita menor cantidad de datos y se ha encontrado que representa bien las palabras raras.

Continuous Bag of Words (CBOW)



CBOW es más rápido y tiene una mejor representación para palabras más frecuentes.

Comparación entre métodos

Característica	Skip-gram	CBOW
Objetivo	Predecir la palabra objetivo **dada una palabra central**	Predecir la palabra de objetivo **dadas las palabras de contexto**
Ventajas	Funciona bien con **palabras infrecuentes**. Mejor representación de **relaciones semánticas**.	**Más rápido en entrenamiento.** **Mejor representación de palabras frecuentes** y sus relaciones sintácticas.
Desventajas	**Más lento en entrenamiento.** **Peor representación de palabras frecuentes**	**Peor representación de palabras infrecuentes** y relaciones semánticas

Muestreo negativo

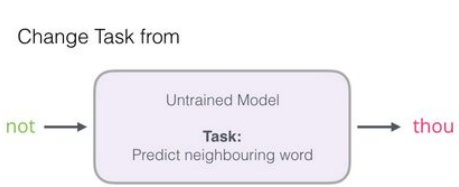
- Debido a que la operación softmax ha considerado que la palabra de contexto podría ser **cualquier palabra en el diccionario** V, la función de pérdida mencionada anteriormente incluye en realidad una **suma tan grande como el mismo diccionario**.
- Para diccionarios más grandes con cientos de miles o incluso millones de palabras, la sobrecarga para calcular cada gradiente puede ser demasiado alta.

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)} \Leftarrow$$

- Para reducir tal complejidad computacional, existe un método de entrenamiento aproximado conocido como **muestreo negativo**.

Muestreo negativo

- El muestreo negativo consiste en transformar el problema de clasificación que plantea skip-gram (cuál es la palabra de contexto dada la central) en un **problema de regresión** que me devuelva la **probabilidad de que dos palabras sean vecinas.**
- Este simple cambio transforma nuestra regresión softmax (usada para clasificación multiclase) en una **regresión logística** (usada para **clasificación binaria**) por lo que se vuelve mucho más simple y mucho más rápido de calcular.





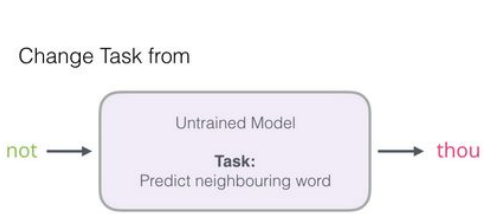
$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}.$$

$$P(D = 1 \mid w_c, w_o) = \sigma(\mathbf{u}_o^\top \mathbf{v}_c),$$

$$\sigma(x) = \frac{1}{1 + \exp(-x)}.$$

Muestreo negativo

- Este cambio requiere que cambiemos la estructura de nuestro conjunto de datos: la etiqueta ahora es una nueva columna con valores 0 o 1. Todos serán 1 ya que todas las palabras que agregamos son vecinas.



input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

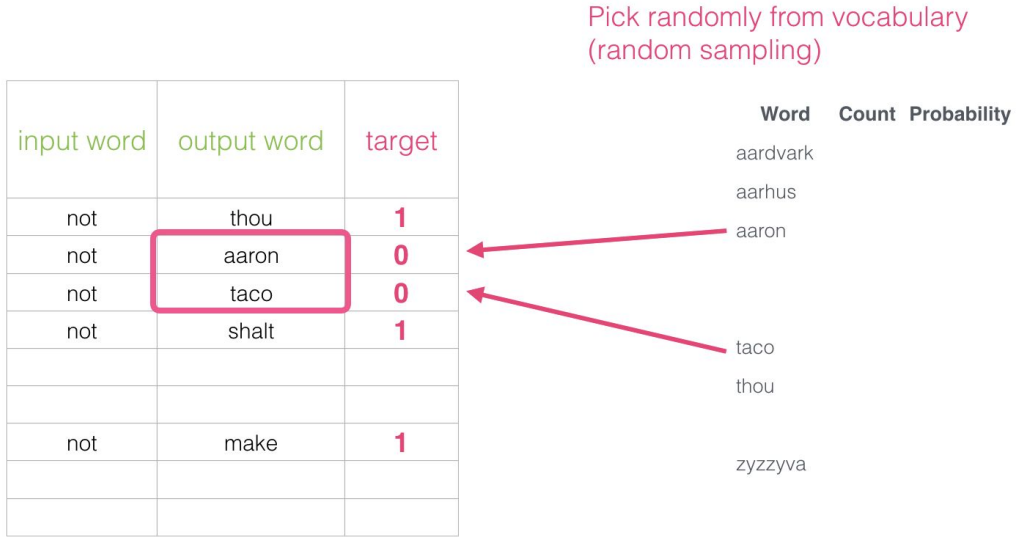


input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

Muestreo negativo

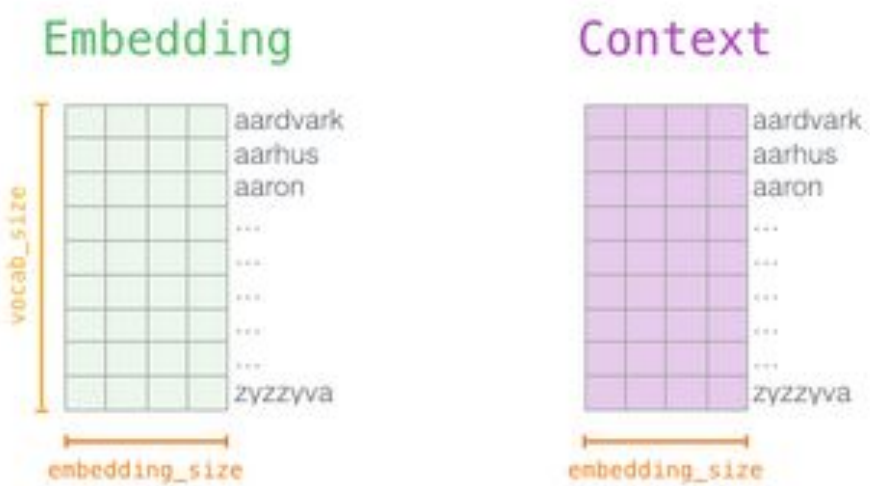
- Esto ahora se puede calcular a una velocidad vertiginosa, procesando millones de ejemplos en minutos. Pero hay un cabo suelto: si todos nuestros ejemplos son positivos (etiqueta: 1), nos abrimos a la posibilidad de un modelo sabelotodo que **siempre devuelve 1**, logrando un 100% de accuracy, pero **sin aprender nada y generando embeddings basura**.

- Para impedir esto, debemos introducir **muestras negativas** en nuestro conjunto de datos: **muestras de palabras que no son vecinas**.
- Nuestro modelo necesita devolver 0 para esas muestras. Ahora, ese es un desafío que el modelo tiene que trabajar duro para resolver, pero aún a una velocidad increíblemente rápida.



Entrenamiento

Al comienzo de la fase de entrenamiento, creamos dos matrices: una **matriz de embedding** y una **matriz de contexto** (los vectores **v** y **u** que vimos antes). Estas dos matrices tienen un embedding de palabra central y de contexto respectivamente para cada palabra en nuestro vocabulario.

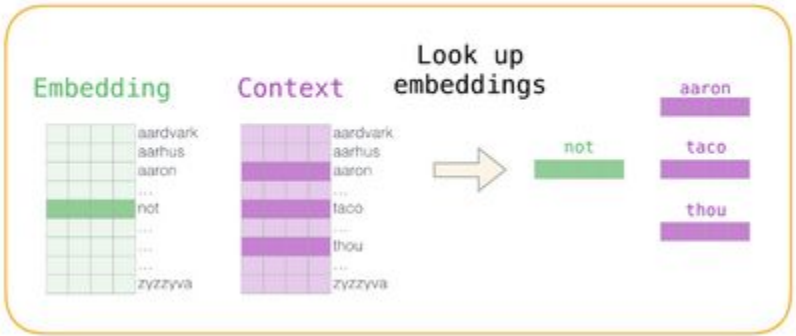


Entrenamiento

Estas dos matrices funcionan como **capas que transforman los índices de token en vectores**. Para cada ejemplo de entrenamiento hacemos pasar los índices de las palabras centrales y de contexto por sus respectivas capas y obtenemos sus embeddings.

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...



Entrenamiento

Luego se **calcula el producto punto** entre el vector de la palabra central por cada uno de los vectores de contexto y negativos para obtener una medida de qué tan similares son (dado que el producto punto es proporcional al coseno).

input word	output word	target	input • output
not	thou	1	0.2
not	aaron	0	-1.11
not	taco	0	0.74

Entrenamiento

Por último, se calcula la **función de pérdida** (que habíamos definido como la **sigmoidea** y eso se utiliza para modificar los parámetros.

La particularidad del entrenamiento de embeddings es que **los embeddings que queremos obtener como salida son los mismos parámetros que estamos entrenando**.



En el caso de word2vec, los embeddings finales se obtendrán de la matriz de palabras centrales, mientras que la matriz de contexto es descartada.

Referencias

- Curso NLP de Instituto Humai. Clase 3, Embeddings
- Dan Jurafsky and James H. Martin. Speech and Language Processing (3rd ed. draft). Chapter I. Part 6: Fundamental Algorithms. Vector Semantics and Embeddings